

UNIVERSITÀ
DEGLI STUDI
DI MILANO

Workshop: Programming Lab

HUMAN-CENTERED ARTIFICIAL INTELLIGENCE

Lecture 3: Conditional statements- Logical operators- Try/except

Dr. Fatme Hachem

06/10/2025

Conditional Statements

Let’s have a look again at this code

```
x=2
y=5
if x<y:
    print("y is greater than x")
elif x==y:
    print("x is equal to y")
else:
    print("x is greater than y")
```

Conditional Statements

Let’s have a look again at this code:

What is going on? What does the «if, elif, else» do?

```
x=2
y=5
if x<y:
    print("y is greater than x")
elif x==y:
    print("x is equal to y")
else:
    print("x is greater than y")
```

Conditional Statements

Conditional statements are an essential part of programming.

They are used to control the flow of the program by executing certain blocks of code only when specific conditions are met.

The "**if**" statement is used to check if a certain condition is true, and if it is, the code inside the block following the "if" statement is executed.

If the condition is false, the code block is skipped.

We can also use "**elif**" to check for other conditions, and "**else**" as a backup plan if none of the conditions are true.

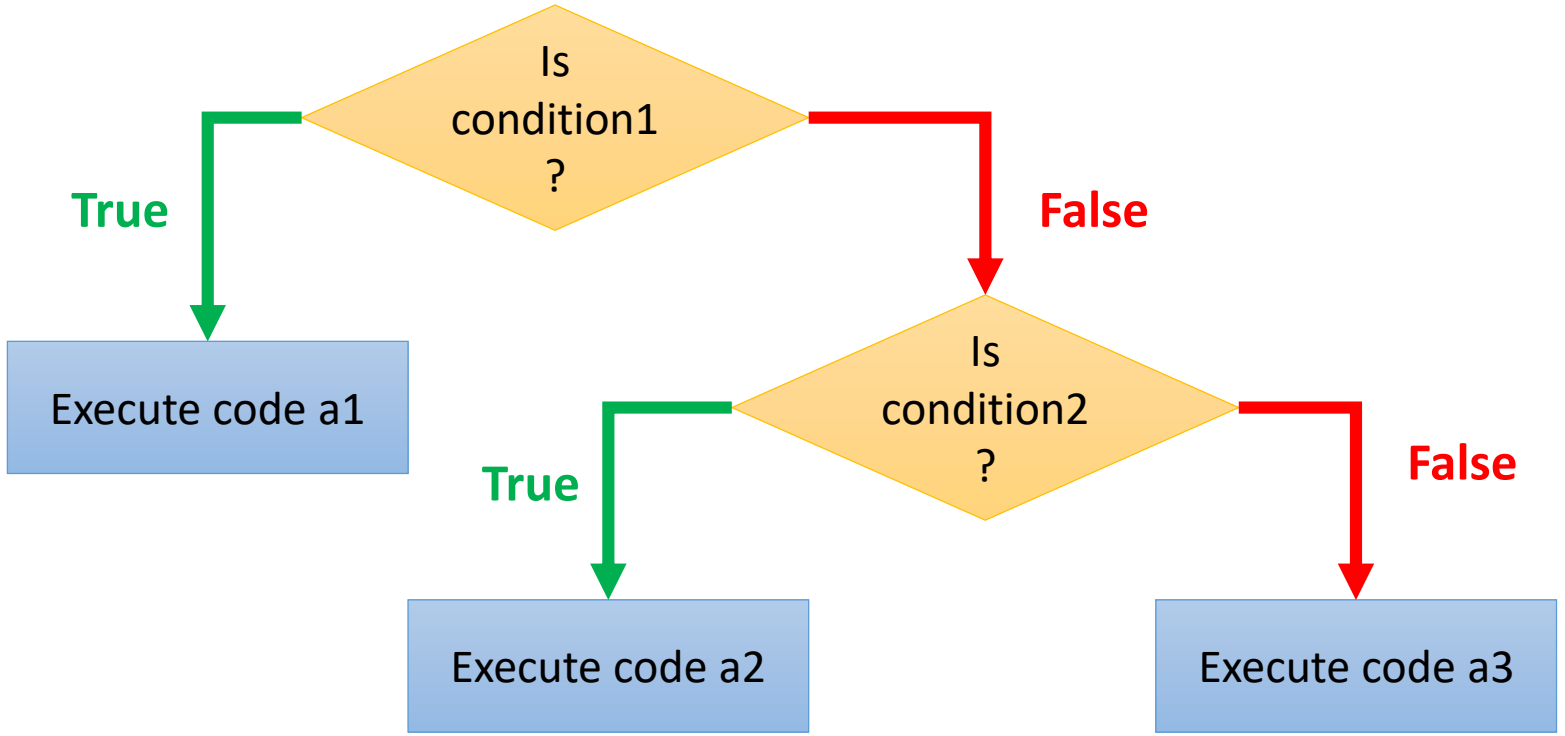
Conditional Statements

```
if condition1:
    code a1

elif condition2:
    code a2

else:
    code a3

code b
```





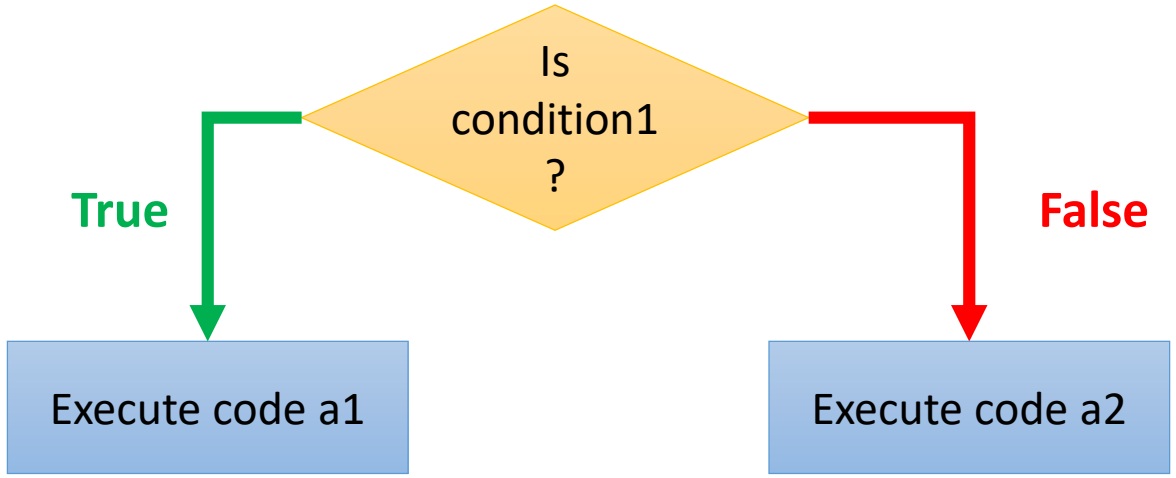
Code b is going to be executed anyways

Conditional Statements

```
if condition1:
    code a1

else:
    code a2

code b
```

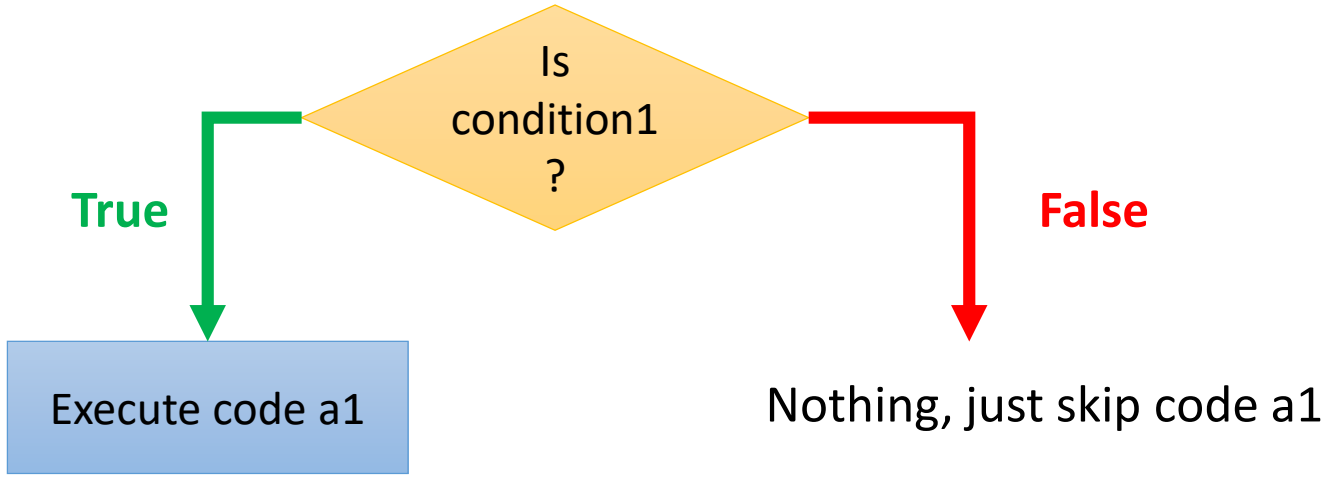


⚠ Code b is going to be executed anyways

Conditional Statements

```
if condition1:
    code a1

code b
```



⚠ Code b is going to be executed anyways

Conditional Statements

Try the following code:

```
my_color= input('select a color from: red, yellow and green: ')

if my_color== 'red':
    print('STOP!')

elif my_color=='yellow':
    print('Slow down')

else:
    print('GO!')
```

Conditional Statements

What if we add a new condition, taking into consideration that user may enter a value different than those suggested?

```
my_color= input('select a color from: red, yellow and green: ')

if my_color== 'red':
    print('STOP!')

elif my_color=='yellow':
    print('Slow down')

elif my_color=='green':
    print('GO!')
else:
    print('Please, select a color from the suggested list')
```

So, we can add as many conditions as needed, by using “elif”

Conditional Statements- Boolean expression

```
if my_color== 'red':
    print('STOP!')
```

The condition is a Boolean expression => asks a question and the response is True or False.

It uses comparison operators:

- < is it less?
- <= is it less or equal?
- > is it greater?
- >= is it greater or equal?
- == is it equal?
- != is it NOT equal?



Attention!
'==' is used as operator in a Boolean expression
'=' is used to assign a value to a variable

Conditional Statements

What is the output of each code?

```
x= 7

if x<10:
    print('less than 10')
elif x>6:
    print('greater than 6')
elif x>4:
    print('greater than 4')
```

```
x= 7

if x<10:
    print('less than 10')
if x>6:
    print('greater than 6')
if x>4:
    print('greater than 4')
```

Conditional Statements- attention to the indentation

What is the output of each code?

```
x= 7

if x<10:
    print('less than 10')
elif x>6:
    print('greater than 6')
print('your number is 7')
```

```
x= 7

if x<10:
    print('less than 10')
elif x>6:
    print('greater than 6')
    print('your number is 7')
```

Conditional Statements

What's wrong in the following code?

```
my_number=input('Enter a number from 0 to 10: ')
my_number=int(my_number)

if my_number=10:
    print('Too high!')
```

Conditional Statements- Are all statement useful?

For which values of “my_number” we will obtain the following output:

- Part time
- Full time
- Undefined schedule

```
my_number= float(input('How many hours you work a day? '))

if my_number<=4:
    print('part time')
elif my_number>4:
    print('full time')
else:
    print('undefined schedule')
```

Examples:

1) Write a Python program that takes an integer as input and prints "Even" if the number is even, and "Odd" if the number is odd.

Example 1: solution

1) Write a Python program that takes an integer as input and prints "Even" if the number is even, and "Odd" if the number is odd.

```
#even or odd?

num = int(input("Enter an integer: "))

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Example 2:

2) Write a Python program that takes two numbers as input and prints the larger number.

Example 2: solution

2) Write a Python program that takes two numbers as input and prints the larger number.

```
#which is larger?

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

if num1 > num2:
    print(num1, "is larger.")
elif num2 > num1:
    print(num2, "is larger.")
else:
    print("Both numbers are equal.")
```

Example 3:

3) Write a Python program that takes as input the percentage score of a student, then prints the corresponding grade. Use the following grading scale:

- 90% and above: A
- 80% - 89%: B
- 70% - 79%: C
- 60% - 69%: D
- Below 60%: F

Example 3: solution

3) Write a Python program that calculates and prints the grade for a student based on their percentage score. Use the following grading scale:

- 90% and above: A
- 80% - 89%: B
- 70% - 79%: C
- 60% - 69%: D
- Below 60%: F

```
# determine the grade
percentage = float(input("Enter your percentage score: "))

if percentage >= 90:
    print("Your grade is A.")
elif percentage >= 80:
    print("Your grade is B.")
elif percentage >= 70:
    print("Your grade is C.")
elif percentage >= 60:
    print("Your grade is D.")
else:
    print("Your grade is F.")
```

Example 3: solution

3) Write a Python program that calculates and prints the grade for a student based on their percentage score. Use the following grading scale:

- 90% and above: A
- 80% - 89%: B
- 70% - 79%: C
- 60% - 69%: D
- Below 60%: F

Is there another method?

Example 3: other solution – The “match” statement

Yes! We can simplify that code using **Python 3.10+**’s **match** statement

```
percentage = float(input("Enter your percentage score: "))

match percentage:
    case p if p >= 90:
        grade = "A"
    case p if p >= 80:
        grade = "B"
    case p if p >= 70:
        grade = "C"
    case p if p >= 60:
        grade = "D"
    case _:
        grade = "F"

print("Your grade is ", grade)
```

Example 3: other solution – The “match” statement

The match statement let us compare a **value** against **several patterns** (not just values) and run code depending on which pattern matches.

```
match variable:
    case pattern1:
        # do something
    case pattern2:
        # do something else
    case _:
        # default / fallback
```

Example 3: other solution – The “match” statement

Going back to pur example:

```
percentage = float(input("Enter your percentage score: "))

match percentage:
    case p if p >= 90:
        grade = "A"
    case p if p >= 80:
        grade = "B"
    case p if p >= 70:
        grade = "C"
    case p if p >= 60:
        grade = "D"
    case _:
        grade = "F"

print("Your grade is ", grade)
```

That means:
If the value of ***percentage*** (assigned to p) is greater than or equal to 90, then set grade to 'A'.

Example 3: other solution – The “match” statement

Example:

```
status = 404

match status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unknown Status")
```

In this example, we are matching the value of «status» against the cases where the value is: 200, 404, 500, every other case

Example 3: other solution – The “match” statement

Consider this example, what would you expect the output to be:

```
value = 20

match value:
    case x if x > 10:
        print("Blue")
    case x if x > 15:
        print("Red")
```

Example 3: other solution – The “match” statement

And for the second example:

```
value = 20

match value:
    case x if x > 10:
        print("Blue")
    case x if x > 15:
        print("Red")
```

```
Blue
```

```
value = 20

match value:
    case x if x > 15:
        print("Red")
    case x if x > 10:
        print("Blue")
```

Example 3: other solution – The “match” statement

And for the second example:

```
value = 20

match value:
    case x if x > 10:
        print("Blue")
    case x if x > 15:
        print("Red")
```

```
Blue
```

```
value = 20

match value:
    case x if x > 15:
        print("Red")
    case x if x > 10:
        print("Blue")
```

```
Red
```

the order of your case statements absolutely matters!
And once one matches, the entire match block is done

Example 4:

4) We have a school that offers scholarships to some young students. The grant amount is based on the age and the grade of the student.

The student has to be younger than 21 years old to be eligible for the scholarship.

Then according to his GPA (1-5) the percentage of the fees discount is calculated:

- if he has a GPA of 5 then he gets a 100% fees discount.
- If he has a GPA of 4 then he gets 80%
- If he has a GPA of 3 then he gets 50%
- If he has a GPA less than 3 then he gets no discount.

Write a program in python that asks the student about his age and GPA (1-5), then tells him whether he is eligible for a scholarship and if yes, the discount percentage he gets.

Example 4: solution

4)

```
# Input
age = int(input("Enter your age: "))
gpa = int(input("Enter your GPA (1-5): "))

# Check eligibility based on age
if age <= 21:
    # If the person is 18 or older, check grade
    if gpa == 5:
        print("You are eligible for 100% scholarship.")
    elif gpa == 4:
        print("You are eligible for 80% scholarship.")
    elif gpa == 3:
        print("You are eligible for 50% scholarship.")
    else:
        print("Your are not eligible for any scholarship.")
else:
    # If the person is above 21, they are not eligible for any course
    print("You are not eligible for any scholarship due to your age.")
```

Logical operators: AND

The “**and**” operator is used to combine two or more conditions in an if statement.

It is necessary to have **ALL** the conditions connected by “and” to be True => entire expression is True.

If any one condition is False => the entire expression is False.

```
#AND operator

age = 12
grade = 4

if age >= 10 and grade >= 3:
    print("You are eligible for the program.")
else:
    print("You are not eligible.")
```

Logical operators: OR

When using the “**or**” operator, it is sufficient to have **at least one** of the conditions to be True => the entire expression is True.

Only in the case where all conditions are False, the entire expression becomes False.

```
#OR operator

is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("You can have fun")
else:
    print("Go study!")
```

Logical operators: NOT

The “**not**” operator is used to negate (reverse) the value of a single condition.

```
is_raining = True

if not is_raining:
    print("You can go outside.")
else:
    print("Stay indoor!")
```

Logical operators- combined

These operators can be combined to create more complex conditions.

⚠ BUT, don't forget to **use parentheses ()** to group conditions and clarify the order of evaluation!

```
weather = input("Is the weather sunny or rainy? : ")
indoor_option = input("Is the place indoor or outdoor? : ")
transportation_available = input("Is there available transportation? yes or no: ")

# Determine whether you can go to the event based on the conditions

if (weather == 'sunny' or (indoor_option == 'indoor' and transportation_available == 'yes')):
    print("You should go to the event!")
else:
    print("It's better to stay home this time.")
```

Example 5:

5) Write a program that asks a student about his age and his GPA (1-5), and tells him if he’s eligible for a course or not.

A student is eligible for a course if he’s older than 10 years old and having a GPA higher or equal to 3. If he’s older than 15 years, he’s automatically eligible for that course.

Example 5: solution

5) Write a program that asks a student about his age and his GPA (1-5), and tells him if he's eligible for a course or not.

A student is eligible for a course if he's older than 10 years old and having a GPA higher or equal to 3. If he's older than 15 years, he's automatically eligible for that course.

```
age = 15
grade = 4

if (age >= 10 and grade >= 3) or age >= 15:
    print("You are eligible for the program.")
else:
    print("You are not eligible.")
```

Try & Except

Sometimes some errors may happen during the execution of a program.

What to do in this case?

When you suspect that a particular piece of code may cause an error, you can place that code inside a **try block**.

If an error occurs within the try block, Python immediately jumps out of the try block and executes the code inside the corresponding **except block**.

```
try:
    #code that we suspect
    #it may raise exceptions
except:
    #code that will be excecuted
    #if the exception we suspected
    #occured
```

Try & Except

Examples on possible exceptions:

- **ValueError**: when a function receives an argument of the correct type but with an invalid value

```
try:
    int('abc')
except ValueError:
    print('invalid value')
```

- **ZeroDivisionError**: When we try to divide a number by zero

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Division by zero is not possible!")
```

Try & Except

Examples on possible exceptions:

- **TypeError**: when an operation is performed on an object of inappropriate type.

```
try:
    result = 10 + "hello"
except TypeError:
    print("Inappropriate types!")
```

- **FileNotFoundError**: when an attempt to open a file fails.

```
try:
    with open("file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found!")
```

Try & Except

We can specify the type of exception we are suspecting to happen, or keep it general without specifying any type in particular. In the latter case, any exception will be handled.

Attention, if you specified the type of exception, and an error of other type occurs, then the code will crash (the error will not be handled).

```
try:
    result = 10 + "hello"
except TypeError:
    print("Inappropriate types!")
```



Error will be handled, no crash

```
try:
    result = 10 + "hello"
except:
    print("Inappropriate types!")
```



Error will be handled, no crash

```
try:
    result = 10 + "hello"
except FileNotFoundError:
    print("Inappropriate types!")
```



Code will crash

Try & Except

We can also use multiple different “except” block for the same “Try”.

```
try:
    result = 10 + "hello"
except TypeError:
    print("Inappropriate types!")
except:
    print("Something went wrong, who knows what!")
```

Try & Except

So…using try & catch will prevent the code from crashing…cool! Why don’t we use it everywhere?! 😎

Try & Except

So…using try & catch will prevent the code from crashing…cool! Why don't we use it everywhere?! 😎

The **Try & except** blocks are powerful tools for handling exceptions in Python, HOWEVER they should be used wisely, as there are some drawbacks to consider:

- Try & except blocks can have a slight performance overhead, especially if an exception is raised frequently. It's generally more efficient to structure your code in a way that avoids unnecessary use of exception handling for expected situations.
- Silent Errors: If you use a broad except block without specifying the type of exception, it can catch unexpected errors that you might not have anticipated. This can make it difficult to diagnose the root cause of the problem.
- Confusing Code Flow

Example 6:

Write a python code that asks the user to enter two numbers, and returns the result of dividing the first by the second.

Make sure to consider potential errors that may occur and handle them appropriately.

Example 6: a solution:

Write a python code that asks the user to enter two numbers, and returns the result of dividing the first by the second.

Make sure to consider potential errors that may occur and handle them appropriately.

```
try:
    num1 = float(input("Enter a number the numerator: "))
    num2 = float(input("Enter a number the denominator: "))
    result = num1 / num2
    print("Result: ", result)

except ZeroDivisionError:
    print("Error: Division by zero is not allowed!")
except ValueError:
    print("Error: Invalid input! Please enter a number.")
```